# The Whole Journey

A scene-by-scene explainer of what this coding agent does and how it does it, following one task from the first command-line flag to the final answer.

**What this app does, and how it does it — a scene-by-scene explainer for the animated page.**

---

### The one-line version

> A coding agent is an **LLM in a loop with tools and a fence**.

You give it a task. It looks around, asks the model what to do next, runs the tool the model names, feeds the result back, and repeats. When the model stops asking for tools and answers, the run is over.

Everything else in this program exists to make that loop **safe, observable and testable**.

**The running example:** create a small script, run it, edit it, run it again.

---

### The journey in one picture

**THE WHOLE JOURNEY** · *text*



CONTENTS

# What is in this explainer


00 · The cast 4

01 · The idea 5

02 · Boot: the CLI builds a small world 6

03 · The request is assembled 7

04 · The loop: seven beats per step 8

05 · The model call 9

06 · The reply: answer or action 10

07 · The dispatcher 11

08 · The eight tools 12

09 · The fences 13

10 · The hooks 14

11 · The memory 15

12 · The session 16

13 · The demo mode 17

14 · The end 18

15 · The journey in one table 19

16 · Why it is built this way 20


Page numbers refer to the folio at the foot of each page.

# 00 · The cast

| MODULE | ITS JOB IN ONE SENTENCE |
|---|---|
| `cli.js` | The front door: flags, banner, one-shot or session, exit codes. |
| `config.js` | Finds the nearest `.env`, merges it with the shell and flags, and resolves the key, model, workspace and budgets. |
| `workspace.js` | Says where relative reads start, and what write paths are out of bounds. |
| `agent.js` | The loop. One exported function, `runTask`. |
| `llm.js` | One model request over HTTP, and sanitising the reply. |
| `prompt.js` | The standing rules, plus the block of long-term memory. |
| `tools.js` | Eight tool schemas, eight handlers, and the dispatcher that connects them. |
| `sandbox.js` | The write fence and the command-line scanner. |
| `hooks.js` | Nine named moments where outside code can watch, rewrite or block. |
| `memory.js` | A markdown file that survives the process. |
| `heartbeat.js` | Prints "still waiting" so a slow model never looks like a frozen program. |
| `demo.js` | A fake model that replays a fixed script, so the loop can run with no key and no network. |
| `ui.js` | Colours, labels and the banner. Cosmetic only — it never changes behaviour. |

> **The important split:** `agent.js` is the agent. Everything else is either a dependency it receives, a boundary it respects, or a way of looking at it.

# 01 · The idea

An ordinary program calls functions. An agent cannot: the model has no hands.

So the harness gives it a language:

- the model writes **text**;
- some of that text is a **tool call** — a name and JSON arguments;
- the harness runs the tool and returns a **string**;
- the model reads the string and decides what to do next.

The model never touches the filesystem, the shell or the network. It only asks. The harness is the part that acts, checks and refuses.

> **The loop is the agent.** Remove the loop and you have a chatbot that can only talk. Keep it and the model can look, act, observe and try again.

---

# 02 · Boot: the CLI builds a small world

Before the model is asked anything, `cli.js` assembles the world the run will live in.

1. **Read the flags.** `--demo`, `--model`, `--max-steps`, `--yes`, `--no-memory`, `--hooks`, `--memory`, and the task itself.
2. **Resolve the config.** `config.js` looks for the nearest `.env` (this package first, then up the tree), merges it with the shell, and applies defaults. `.env` deliberately beats the shell, so a stale key in `~/.bashrc` cannot silently win.
3. **Build the pieces.** A workspace root, a memory store, a sandbox fence, a hook registry, and a `chatFn`: - live: one HTTP request to OpenRouter; - demo: a scripted fake model.
4. **Print the banner.** Workspace, sandbox, model, key fingerprint, env file, command policy, timeout, tools, memory, hooks. The banner is a receipt: it says which world this run is in.
5. **Choose the shape of the run.** - a task was given → run it once and set an exit code; - no task, but a terminal → start an interactive session; - no task and no terminal → stop with exit code 2, because nobody can answer the prompt.

Two side doors need no model at all: `--hooks` explains the lifecycle, and `--memory` prints the memory file and the exact block the prompt would receive.

> **On screen:** a terminal banner builds line by line, like a cockpit check before takeoff. Every line is a dependency the loop is about to receive.

---

# 03 · The request is assembled

`runTask` starts by building the one object the model will ever see: `messages`.

**SRC/AGENT.JS** · *javascript*

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
```

Three layers, in order:

| LAYER | WHERE IT COMES FROM | WHY IT IS THERE |
|---|---|---|
| **System prompt** | `prompt.js` + `memory.js` | How to work, what the tools are for, and what earlier runs learned. |
| **History** | the previous task in this session | So "now add tests" knows what was just built. The old system message is dropped and rebuilt. |
| **Task** | you | The one thing this run is trying to do. |

Separately, every request also carries the **tool schemas** from `tools.js`: the model's menu of possible actions.

The model has no hidden memory. If something is not in `messages`, the system prompt or the tool schemas, it does not exist for this step. That is why this array is the agent's entire working state.

> **On screen:** three translucent layers stack into a single envelope. The tool schemas hover beside it as a separate menu.

# 04 · The loop: seven beats per step

The heart of the app is one `for` loop in `agent.js`.

| BEAT | WHAT HAPPENS |
|---|---|
| **1. Announce** | Print `step 3/12` and arm the heartbeat. |
| **2. Ask the model** | Send `messages` plus the tool schemas. Wait for one reply. |
| **3. Check for truncation** | If the reply was cut off by the length limit, do not dispatch half a tool call. Ask the model to continue smaller. |
| **4. Append the reply** | Strip it to the fields an agent needs, then push it into `messages`. |
| **5. Answer?** | If there are no tool calls, print the answer and stop. |
| **6. Dispatch** | For each tool call: run it, get a string, append a `role: "tool"` message with the matching `tool_call_id`. |
| **7. Loop** | Go back to beat 1 with a bigger `messages` array. |

If the loop reaches `maxSteps` without an answer, it stops with a step-limit failure. Failures have a lifecycle too.

## The running example

| STEP | THE MODEL ASKS | THE TOOL RETURNS |
|---|---|---|
| 1 | `list_files` the workspace | the current file list |
| 2 | `create_file` `demo/hello.js` | `created demo/hello.js (72 bytes)` |
| 3 | `run_command` `node demo/hello.js` | `exit code 0` + `stdout: Hello ...` |
| 4 | `edit_file` one string | `edited demo/hello.js: 1 replacement, first at line 2` |
| 5 | `run_command` again | `exit code 0` + the new output |
| 6 | *(no tool calls)* | — the model writes the final answer |

Each result becomes a new message, so step 6 can see everything steps 1–5 learned.

> **On screen:** a glowing dot travels around a loop. On each pass, a new message card is added to the stack. When the card says "answer", the dot leaves the loop.

# 05 · The model call

One step is one request. `llm.js` sends an OpenAI-compatible POST:

**ONE MODEL REQUEST** · *text*

```
POST /chat/completions
{
  model,
  messages,        // system + history + task + everything so far
  tools,           // the eight tool schemas
  max_tokens: 2048
}
```

Three practical details:

- **The request is bounded.** `max_tokens: 2048` keeps each reply small, and the timeout stops a stalled provider from hanging the run.
- **The heartbeat is armed.** Every five seconds the console prints "still waiting for the model". A slow call must not look like a frozen program.
- **The reply is sanitised.** Only `content` and `tool_calls` are kept. Private reasoning fields are dropped so they are not re-sent on every later step.

There is no streaming. The loop asks for a complete reply, then decides what to do with it. Streaming would improve the feel of the console, but it would not change the shape of the agent.

> **On screen:** an envelope leaves for a cloud labelled "model", a pulsing wait line runs, and a reply envelope comes back with `finish_reason` stamped on it.

# 06 · The reply: answer or action

The model's reply is one of two things:

**A. An answer.** No `tool_calls`. The loop prints the content and returns:

**THE SHAPE OF AN ANSWER** · *javascript*

```javascript
{ ok: true, answer: "...", messages, steps }
```

**B. One or more tool calls.** Each call has:

**THE SHAPE OF A TOOL CALL** · *json*

```json
{
  "id": "call_abc",
  "type": "function",
  "function": {
    "name": "read_file",
    "arguments": "{\"path\":\"src/agent.js\",\"limit\":40}"
  }
}
```

The arguments are a **JSON string**, written by the model. They are not trusted. They may name a tool that does not exist, use the wrong type, or ask for a path outside the workspace. That is the dispatcher's problem.

Every tool call **must** get a matching tool message with the same `id`, even when the tool failed. A provider rejects the next request if a call is left unanswered. That is why the append happens inside the same loop that runs the tool.

> **On screen:** the reply card flips. Heads: a green answer card floats away. Tails: a handful of action cards fall into a queue labelled "to dispatch".

# 07 · The dispatcher

*(In plain terms: the dispatcher is the function that takes the model's tool call, checks the arguments, runs the matching tool, and returns a string.)*

`runTool(name, rawArgs, ctx)` is the door between the model's intention and the real world. It does four things, in order:

1. **Look the tool up.** Unknown name? Return a helpful string listing the tools that do exist.
2. **Validate the arguments.** Parse the JSON, check it is an object, check each provided value against the schema's type. A model sending `"limit": "20"` gets `ERROR: "limit" must be a integer, got string`.
3. **Run the handler.** Every handler receives the workspace, the arguments and the run context.
4. **Catch everything.** A thrown error never escapes the loop: - `ForbiddenError` → `FORBIDDEN: ...` - `PathError` → `ERROR: ...` - anything else → `ERROR: <code>: ...`

The rule that makes the whole loop work: **the dispatcher always returns a string**.

Success, denial, bad arguments, an unexpected exception — all of it becomes text the model reads on the next step. A thrown error would end the run; a returned error is an observation the model can recover from.

> **On screen:** the action card goes through a four-stage machine. The output slot always produces a text strip, even when the strip is red.

---

# 08 · The eight tools

| TOOL | WHAT IT DOES | SAFETY |
|---|---|---|
| `list_files` | Lists files and folders, with depth and a line cap. | Read-only. |
| `read_file` | Reads a slice of a file with line numbers. | Read-only; paginated. |
| `create_file` | Creates a new file; refuses to overwrite. | Write fenced by the sandbox. |
| `write_file` | Creates or overwrites a file. | Write fenced by the sandbox. |
| `edit_file` | Replaces an exact `old_string` with `new_string`. | Write fenced; refuses ambiguous matches. |
| `run_command` | Runs a shell command and reports the exit code, stdout and stderr. | Policy + deny-list + shell-write scan + env allow-list. |
| `remember` | Saves a durable note to `memory.md`. | Capped, deduplicated, tagged. |
| `recall` | Searches `memory.md`. | Read-only. |

Two asymmetries are deliberate:

- **Reads are open; writes are fenced.** An agent that cannot read a library, a log or a doc outside its folder is useless. An agent that can write anywhere is dangerous.
- **Actions return small strings.** `create_file` says "created, 72 bytes", not the whole file. `edit_file` says "one replacement at line 2", not the new source. The model already knows what it wrote; re-sending it would waste context.

> **On screen:** eight tool cards fan out. Each has a small lock icon that is green for reads, amber for writes, red for shell commands.

# 09 · The fences

The model is not trusted, so the harness puts checks at the boundary.

**The sandbox (writes).** `sandbox.js` checks every save, edit and delete before touching disk. It refuses:

- paths that climb out with `..`,
- absolute paths outside the workspace,
- `~`, `%VAR%`, UNC paths,
- home and system folder names,
- anything that resolves outside the root.

A refusal prints one bold `FORBIDDEN` line and returns the same sentence to the model. Nothing is written.

**The command policy (shell).** `CMD_POLICY` is `allow`, `ask` or `deny`. In `ask`, the user confirms each command. Independently of the policy, certain commands are always refused: recursive deletes of root paths, disk formatting, shutdown, force-push, `curl | sh`, fork bombs.

**The shell-write scan.** A command can do what a file tool cannot. Before executing, `sandbox.js` tokenises the command line and looks for visible write targets: `>`, `>>`, `copy`, `move`, `xcopy`, `del`, `rm`, PowerShell destinations. If a target resolves outside the workspace, the command is refused.

**The environment allow-list.** Child processes get 13 environment variables, not `process.env`. Without this, `node -e "console.log(process.env.OPENROUTER_API_KEY)"` would put the key straight into the transcript.

None of this is real isolation. A script can still walk out; a container is the answer for that. The fences are **best-effort guardrails** that make the common accidents loud and the obvious attacks impossible.

> **On screen:** a fence surrounds the workspace. Green paths lead to reads outside. Red paths bounce off with a `FORBIDDEN` stamp before they reach disk.

# 10 · The hooks

The loop is deliberately fixed. But a fixed loop is not extensible, so the app has **nine named moments** where outside code gets a turn:

| MOMENT | WHEN | WHAT IT CAN DO |
|---|---|---|
| `SessionStart` | before the first request | observe, or refuse to start |
| `UserPromptSubmit` | as the task enters | rewrite the task, or block it |
| `PreModel` | before every model call | observe the request (measure tokens here) |
| `PostModel` | after every reply | observe the answer or tool call |
| `PreToolUse` | before every tool call | **block** the call |
| `PostToolUse` | after every tool call | **replace** the result the model reads |
| `Stop` | the model answered | observe, summarise, notify |
| `StepLimit` | the step budget ran out | observe, escalate |
| `SessionEnd` | always, finished or not | clean up, write a transcript |

A hook returns a small object:

- nothing → observe only;
- `{ note }` → an observation for the console;
- `{ task }` → rewrite the task;
- `{ replace }` → rewrite a tool result;
- `{ block, reason }` → refuse the call; the model is told why.

A hook that throws is reported and ignored. Extension code must never be able to break the loop's invariants — above all, every tool call still gets a result.

The default run attaches a **trace hook** to every moment, which is why the console prints lines like `[HOOK:PreToolUse] before every tool call`. `--no-hooks` silences them; `--hooks` explains them.

> **On screen:** a timeline runs alongside the loop. Nine beacons light up in sequence. One beacon flashes red and a tool call stops before it runs.

# 11 · The memory

`messages` dies with the process. `memory.md` does not.

The file lives beside the code, not inside the sandbox. The agent reaches it only through two tools, so it cannot be clobbered by a normal file edit:

**ONE BULLET PER MEMORY** · *text*

```
- [fact] The demo script lives at demo/hello.js. _(2026-10-28)_
- [gotcha] This repo uses ESM, so require() fails. _(2026-10-30)_
```

`[tag]` is `fact`, `pref`, `fix` or `gotcha`; the date is when it was learned. It is deliberately the least clever format that still parses: a human can read it, diff it, or delete a wrong line by hand.

There are two access paths:

| PATH | WHAT IT DOES | COST |
|---|---|---|
| **Eager —** `digest()` | The newest 12 entries, capped at 1,200 characters, appended to the system prompt at the start of each task. | Fixed. It cannot grow with the file. |
| **Lazy —** `recall` | Searches the whole file on demand. | One tool call, only when needed. |

The digest ends by saying what it left out: "(9 older memories not shown — search `src/memory.md` with the `recall` tool)". That is the difference between "this is all I know" and "this is what fits; ask for the rest".

Writes are bounded too: duplicates refresh the date, one entry is capped at 300 characters, one call at 2,000, and the file holds at most 60 entries. The oldest are dropped.

Why a file and not a database? Because "why did it remember that?" should be answerable with `cat`, not a log hunt. No embeddings, no vector search, no automatic summarising — the smallest thing that actually persists.

> **On screen:** a file icon sits outside the workspace fence. At task start, a small stack of note cards flies from the file into the system prompt. Later, a `recall` call opens the whole file.

# 12 · The session

A task is one `runTask` call. A session is tasks, one after another, without leaving the program.

Four things make that work:

1. **The step budget resets by construction.** Each `runTask` starts its `for` at step 1, so the second task gets its own 12 steps.
2. **The conversation is carried, the system prompt is not.** The CLI keeps the last 24 messages and passes them as `history`. The system message is rebuilt, so a memory saved by task one is already visible in task two.
3. **The history cut lands on a `user` turn.** A provider rejects a history that starts with a tool result or ends with an unanswered tool call, so `trimHistory` walks forward to a safe boundary.
4. `/new` **clears the conversation, not the project.** Files and `memory.md` stay. `/quit` ends the session.

A one-shot run does the same work once, then turns the result into an exit code:

| CODE | MEANING |
|---|---|
| `0` | the model answered |
| `1` | the request failed (bad key, network, provider error) |
| `2` | no task, and no terminal to ask for one |
| `3` | the step limit was reached before an answer |

> **On screen:** a session timeline shows task cards. Each card runs the same loop. A dashed line carries the last part of one card's conversation into the next, while the system prompt is rebuilt fresh at the top.

# 13 · The demo mode

`--demo` replaces exactly one thing: the model.

`demoChat()` is a `chatFn` with the same return shape as OpenRouter. It replays a fixed script of assistant messages:

- steps 1–5: list, create, run, edit, run again;
- steps 6–12: try to escape the sandbox, read outside it, write to the home folder, copy out with the shell, force-push, echo the API key, and fail a command;
- steps 13–14: `remember` and `recall`.

Everything else is real: the loop, the dispatcher, the sandbox, the memory tools, the hooks, the exit codes. The only missing piece is one HTTP POST.

At the end, a **scorecard** reads the run's own tool results back and proves each guard actually held. It is a claim you can check, printed by the program.

> **On screen:** a "scripted model" card drops into the model slot. The loop does not notice. The same journey plays, with red `FORBIDDEN` stamps where the script tries to break out.

---

# 14 · The end

When the model finally answers, the loop:

1. prints the answer under a rule;
2. fires `Stop`;
3. fires `SessionEnd` — always, whatever happened;
4. returns `{ ok, answer, messages, steps }`.

The CLI turns that into an exit code, or — in a session — prints a small summary and asks for the next task.

The workspace keeps the files. `memory.md` keeps the durable facts. The process ends. The next run starts with an empty `messages` array and a memory block built from what survived.

> **On screen:** the loop finishes its last lap, the final answer card settles at the bottom, and the workspace tree shows the new file still there after the terminal closes.

---

# 15 · The journey in one table

| STAGE | MODULE | WHAT HAPPENS | WHAT COMES OUT |
|---|---|---|---|
| 1. Boot | `cli.js`, `config.js` | Flags, env, defaults, key, model, budgets | a configured world |
| 2. Build | `workspace.js`, `memory.js`, `sandbox.js`, `hooks.js` | The folder, the memory file, the fence, the lifecycle | dependencies for the run |
| 3. Assemble | `prompt.js`, `agent.js` | Rules + memory + history + task | the `messages` array |
| 4. Ask | `llm.js` | One model call with tools | a reply with `content` and/or `tool_calls` |
| 5. Decide | `agent.js` | Answer, truncation, or tool calls | stop or dispatch |
| 6. Act | `tools.js` | Validate, run, catch | one string per call |
| 7. Guard | `sandbox.js` | Check writes and commands | allow or `FORBIDDEN` / `DENIED` |
| 8. Observe | `hooks.js` | Nine lifecycle moments | notes, rewrites, blocks |
| 9. Remember | `memory.js` | Eager digest + lazy recall | facts that survive the process |
| 10. Repeat | `agent.js` | Append results and loop | a growing transcript |
| 11. Finish | `agent.js`, `cli.js` | Answer, exit code, session summary | the result |

# 16 · Why it is built this way

1. **The loop is the agent.** Everything else serves one `for` loop. That is what makes the program readable in one sitting.
2. **Messages are the state.** No hidden memory, no scratchpad service. If it is not in `messages`, the model does not know it.
3. **Tools are the only action.** The model asks in JSON; the harness executes and checks.
4. **Errors are observations, not crashes.** A failed edit comes back as text and the model corrects itself on the next step.
5. **The model is not trusted.** Writes are fenced, commands are scanned, the environment is allow-listed.
6. **Autonomy is bounded.** A step limit, output caps, timeouts and memory caps turn runaway behaviour into a visible stop.
7. **Extension points have names.** Nine hooks mean you can add logging, permissions, redaction or compaction without editing the loop.
8. **Dependencies are injected.** The tests run the real loop with a fake model, no key and no network.
9. **Memory is a file, not a mystery.** You can read it, diff it, edit it and delete a wrong line.

---

> **The whole thing in one sentence:** a model asks, a dispatcher acts, a sandbox refuses, a loop repeats, and a file remembers — until the model answers.